# Compe 271 Project

**Bryan Heraz, 5/8/2025, bheraz4127@sdsu.edu**

## 1.Project Goal

The main objective of this project is to demonstrate the effectiveness of cache memory and runtime efficiency by observing various memory access patterns in a C program. Memory hierarchy is a common implementation found in modern CPUs, which include three levels of fast cache memory (L1, L2, L3) and large slow main memory (RAM). The CPU can easily access data if it can be stored within cache memory, causing a cache hit. However if data goes beyond cache limitations, the CPU moves on to the main memory which results in a cache miss, affecting performance.

To analyze this behavior, this project will focus on two elements, memory access patterns (large strides) and array capacity. Cache misses and hits can be determined by measuring execution time. Faster execution means more cache hits, while slow execution time implies cache misses. The first experiment will focus on array capacity and sequential accessibility. Sequential accessibility has good spatial locality since the CPU can predict its next memory location to access. However, if array capacity exceeds cache's size, CPU will start fetching data in main memory and will reduce performance. While this does capture the effect of capacity, it does not demonstrate the benefit of memory access patterns. Therefore, the second experiment focuses on stride access patterns in vectors/arrays. Spatial locality is reduced the larger the stride value, wasting cache lines, leading to cache misses.

Overall, the purpose of this project is to show the significance of cache performance. For the purpose of this project, these experiments were tested with an AMD Ryzen 5 7600X 6-Core processor with cache sizes of 384KB(L1), 6.0MB (L2), and 32.0MB (L3).

## 2.Project Implementation

A simple two vector/arrays (A and B) sum operation was implemented for this project, in which each element is read, computed and stored in the third array (C). This algorithm was

recommended due to its simplicity and highlighting the main aspect of cache behavior and its execution time. To execute this, a C program was created to calculate two arrays sequentially and non-sequentially. Various helpers functions and libraries were created and added to help construct the project. All three arrays were memory allocated using malloc function found in C stdlib.h library. By creating an array() function, array A and B can be filled before times are recorded. Header time.h was implemented to use clock_gettime(), a function that measures performance by using CLOCK_MONOTONIC that is provided by POSIX 2008 version. By creating a get_time() function, clock_time() gets called with CLOCK_MONOTONIC to measure time. After being recorded,the value is stored in timespec address which tv_sec tv_nsec can be accessed with member access operator.

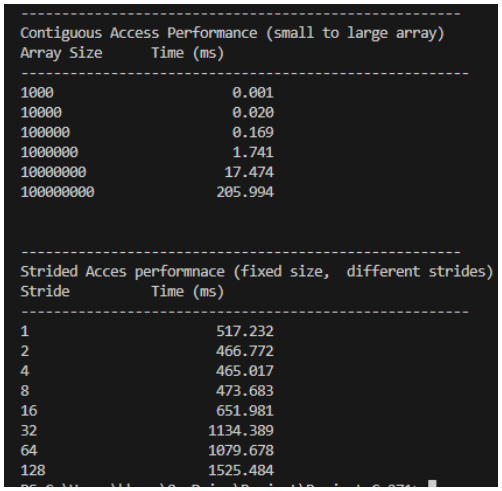
Cache size has been selected based on our CPU cache restriction, which consists of 1000, 10000, 100000,1000000, 10000000, and 100000000 elements. First two values fit in L1 cache, third value fits in L2 cache, fourth in L3, and last 2 values require main memory access. For the purpose of the first experiment, contiguous access is utilized to prioritize the effect of data capacity in cache performance. A contiguous_access() function is implemented to perform vector addition for A and B using a for loop, then return sum in C. This function performs a sequential pattern adding both arrays elements one by one.

In order to perform a non-constiguous algorithm, stride_access() function was implemented to perform vector addition by accessing elements based on its non-contiguous behavior. Each Stride value is stored in a list, located in the main function. To ensure fairness within this experiment, variable OP was given a high value number in order to control the amount of operations performed. If a small loop operation was given, the large stride will have fast execution time since it performs fewer computations. Index is initialized to zero then is

updated within “for loop” using the formula index  = (index + stride ) % size. This formula allows the updated index to be within the size of the array.

Both of these algorithms are tested within the main function. Two for loops are used to test different sizes and strides, stored in “current_size” and “current_stride”. Inside these for loops, get_time() function is called twice to record start time and end time when the algorithm is finished. Last, overall time is calculated and printed in the terminal.

**3.Results & Insight/Conclusion**



```
------------------------------------------------------------
Contiguous Access Performance (small to large array)
Array Size      Time (ms)
------------------------------------------------------------
1000                    0.001
10000                   0.020
100000                  0.169
1000000                 1.741
10000000               17.474
100000000             205.994


------------------------------------------------------------
Strided Acces performnace (fixed size,  different strides)
Stride          Time (ms)
------------------------------------------------------------
1                     517.232
2                     466.772
4                     465.017
8                     473.683
16                    651.981
32                   1134.389
64                   1079.678
128                  1525.484
```

Based on these results, as array size increases, execution time increases. This is due to the fact that if the array size is small, it can fit in cache which therefore gives CPU easy access to the data. As it increases, cache misses occur because the CPU needs to access the main memory. As for non-contiguous behavior, strides values 1-8 stay the same execution time because multiple data is used within the same cacheline. However, after 16 strides, execution increases rapidly. Implying a lot of cache misses. As stride increases, a certain amount of cache lines are skipped while only using one value of each line it went, which wastes cache lines. This algorithm implements bad spatial locality, forcing the CPU to fetch new cache lines very often from memory.